

Guía: Planes lógicos y físicos

Algoritmos de join, selección y ordenamiento de operaciones

1. Transacciones, Concurrency y Triggers

Notación. Modelamos cada transacción como una secuencia de operaciones primitivas: $R_i(X)$ (la transacción i lee el objeto X), $W_i(X)$ (la escribe), C_i (commit) y A_i (abort). Recuerda que dos operaciones *conflictúan* si son de transacciones distintas, tocan el mismo objeto y al menos una es escritura.

Ejercicio 1. Grafo de precedencia y conflict-serializabilidad Para cada schedule, construya el **grafo de precedencia**, determine si es **conflict-serializable** y, en caso afirmativo, entregue un orden serial equivalente.

- (a) S1 = W1(A) R2(A) W2(B) R3(B) W3(C) R1(C)
- (b) S2 = R1(A) W1(A) R2(A) R1(B) W2(A) W1(B) R2(B) W2(B)
- (c) S3 = R1(X) R2(X) R3(Y) W1(X) W2(Y) R1(Y) W3(X)

Sugerencia. Recorra el schedule de izquierda a derecha y agregue una arista $T_i \rightarrow T_j$ cada vez que una operación de T_i aparezca antes de una operación conflictiva de T_j . El schedule es conflict-serializable si y sólo si el grafo es acíclico; cualquier orden topológico es un serial equivalente.

Ejercicio 2. Planificación con localidad de buffer Recuerde por qué el DBMS intercala transacciones en vez de correrlas una a una: mientras una espera I/O, otra avanza. Un beneficio concreto de intercalar es la **localidad de buffer**: si agrupamos varias operaciones sobre la *misma página*, ésta se trae a memoria **una sola vez** en lugar de leerse repetidas veces desde disco.

Modelo de costo (simplificado). Suponga que el buffer pool tiene espacio para **una sola página**. Para leer o escribir una página, ésta debe estar en el buffer; traer una página desaloja la que estaba. Cada vez que la *página activa* cambia, se paga **1 lectura de disco**. Minimizar lecturas $\equiv$ agrupar la mayor cantidad de operaciones consecutivas sobre la misma página.

Tenemos tres transacciones, cada una hace lectura+escritura (read-modify-write) sobre dos páginas:

T1: R1(A) W1(A) R1(B) W1(B)
 T2: R2(B) W2(B) R2(C) W2(C)
 T3: R3(A) W3(A) R3(C) W3(C)

Para cada uno de los siguientes regímenes, construya un schedule **correcto** que respete las reglas del régimen y que **minimice el número de lecturas de disco**. Para cada uno, indique el schedule, el orden de páginas activas y el total de lecturas.

- (a) **Strict 2PL.** Cada transacción adquiere sus locks (S/X) antes de usar cada objeto y **los libera todos sólo en el commit**. Anote las adquisiciones y el commit. Observe que cada par de transacciones comparte una página: ¿puede *intercalar* los accesos de dos transacciones a una misma página bajo strict 2PL? Como pista de la consecuencia: el mejor schedule aquí resulta esencialmente *serial* — encuentre el **orden serial** con menos lecturas.

- (b) **2PL (no estricto).** Ahora una transacción *puede* liberar un lock antes del commit, pero una vez que libera el primero **no puede adquirir ninguno más** (fase de crecimiento, luego de decrecimiento). Aprovechando esta libertad, ¿puede agrupar mejor los accesos a cada página y bajar el número de lecturas respecto a (a)? Muestre el schedule con sus locks.
- (c) **Cualquier schedule conflict-serializable.** Olvídense de los locks: construya el schedule que **minimice** las lecturas de disco y luego **demuestre** que es conflict-serializable dibujando su grafo de precedencia y dando el orden serial equivalente. ¿Cuál es el mínimo *teórico* de lecturas, dado que hay 3 páginas distintas?
- (d) **Comparación.** Ordene los tres regímenes por número de lecturas. ¿Qué le “cuesta”, en I/O, la disciplina de strict 2PL? Conecte esto con por qué strict 2PL —pese a ese costo— es el estándar de producción (recuerde *cascading aborts*: ¿qué pasaría en (b) o (c) si una transacción que ya liberó locks y cuyas escrituras otra alcanzó a leer termina abortando?).

Sugerencia. Una transacción retiene la página exactamente mientras tenga el lock sobre ella. Bajo strict 2PL eso es “hasta el commit”, así que dos transacciones que tocan la misma página no pueden agrupar sus accesos a esa página. En (b)/(c), en cambio, puede llevar todos los accesos a *A* juntos, luego todos los de *B*, luego los de *C*: verifique que el orden de páginas *A...A B...B C...C* es alcanzable y que su grafo de precedencia queda acíclico. En (b), recuerde que se puede *adquirir* el lock de una página y *operar* sobre ella más tarde (operar no es adquirir).

Ejercicio 3. Deadlocks y grafo de espera En un instante dado, el lock manager tiene este estado entre cuatro transacciones:

- T_1 mantiene X(A) y solicita X(B).
- T_2 mantiene X(B) y solicita X(C).
- T_3 mantiene X(C) y solicita X(A).
- T_4 mantiene X(D) y solicita X(C).

- (a) Dibuje el **grafo de espera** (*waits-for graph*): un nodo por transacción y una arista $T_i \rightarrow T_j$ si T_i espera un lock que T_j mantiene.
- (b) ¿Hay deadlock? Identifique el ciclo.
- (c) PostgreSQL no previene deadlocks: los detecta tras `deadlock_timeout` y aborta una *víctima*. ¿Qué transacciones podrían elegirse como víctima para romper el ciclo? Justifique.

Sugerencia. T_4 participa del grafo pero quizá no del ciclo; abortar una transacción que no está en el ciclo no lo rompe.

Ejercicio 4. Triggers La tabla `Cuentas(cid, saldo)` registra saldos, y queremos mantener automáticamente dos invariantes y una bitácora:

- (a) Escriba una función y un trigger en PL/pgSQL que **impida** que cualquier INSERT o UPDATE deje un saldo negativo, abortando la operación con un mensaje claro. ¿Debe ser BEFORE o AFTER? ¿FOR EACH ROW o FOR EACH STATEMENT? Justifique.
- (b) Existe una tabla `Auditoria(id, cid, saldo_viejo, saldo_nuevo, ts)`. Escriba un trigger que registre allí cada modificación de saldo. Use las variables OLD, NEW y, si lo necesita, TG_OP. ¿Por qué este trigger conviene que sea AFTER?

- (c) Explique qué pasa con el efecto del trigger de (b) si la transacción que disparó el UPDATE hace ROLLBACK. Relaciónelo con la atomicidad: ¿el registro de auditoría persiste?
- (d) Mencione un caso en que un trigger FOR EACH ROW sobre un UPDATE masivo (millones de filas) sea una mala idea, y qué alternativa usaría.

Sugerencia. Un trigger se ejecuta *dentro* de la misma transacción que lo disparó: si la transacción aborta, los efectos del trigger se deshacen junto con todo lo demás. BEFORE sirve para validar/normalizar o cancelar; AFTER para propagar a otras tablas una vez que la fila ya cambió.

2. Recuperación de Fallas

Recordatorio de notación.

- **Undo logging:** el registro de update es $\langle T, X, t \rangle$, donde t es el valor **antiguo** de X . Regla: las páginas de datos van a disco *antes* de $\langle \text{COMMIT } T \rangle$.
- **Redo logging:** el registro es $\langle T, X, v \rangle$, donde v es el valor **nuevo**. Regla: el log (incluido $\langle \text{COMMIT } T \rangle$) va a disco *antes* que las páginas de datos.
- **Undo/Redo:** el registro es $\langle T, X, t, v \rangle$ (ambos valores); no se impone orden entre escribir datos y escribir el commit.
- En todos los casos vale el principio **WAL**: el registro de log de una modificación está en disco antes que la modificación misma.

Ejercicio 1. Recovery con Undo Logging Tras un crash, el log en disco contiene (los valores son los **antiguos**):

```
<START T1>
<T1, A, 5>
<START T2>
<T2, B, 10>
<T1, C, 12>
<COMMIT T1>
<T2, D, 20>
<START T3>
<T3, E, 30>
<-- CRASH
```

- (a) Recorra el log *de atrás hacia adelante* y diga, para cada registro de update, si se deshace y a qué valor queda el objeto. Indique qué transacciones se consideran *completas*.
- (b) ¿Qué registros $\langle \text{ABORT } T \rangle$ se agregan al final del log?
- (c) Suponga que el sistema vuelve a fallar *en medio* de la recuperación, justo después de deshacer $\langle T2, D, 20 \rangle$. Explique por qué volver a correr el algoritmo completo da el mismo resultado (idempotencia).

Sugerencia. Una transacción es *completa* si tiene $\langle \text{COMMIT} \rangle$ o $\langle \text{ABORT} \rangle$ en el log. Sólo se deshacen las escrituras de transacciones **no** completas.

Ejercicio 2. Recovery con Redo Logging El mismo escenario, pero ahora con **redo logging** (los valores son los **nuevos**):

```
<START T1>
<T1, A, 7>
<START T2>
<T1, C, 9>
<COMMIT T1>
<T2, B, 11>
<START T3>
<T3, E, 25>
<COMMIT T3>
<T2, D, 30>
    <-- CRASH
```

- (a) ¿En qué dirección se recorre el log con redo? Liste el conjunto de transacciones *confirmadas*.
- (b) Para cada registro de update, diga si se **re-hace** (y a qué valor) o se ignora.
- (c) ¿Qué registros **<ABORT T>** se agregan al final?
- (d) Compare con el Ejercicio 1: en undo, antes del commit los datos ya están en disco; en redo, no necesariamente. ¿Cuál de los dos hace el **COMMIT** más rápido y por qué?

Sugerencia. En redo, sólo re-hacemos las escrituras de transacciones **confirmadas**; las no confirmadas se ignoran porque sus datos nunca pudieron llegar a disco antes del commit (regla R1).

Ejercicio 3. Undo/Redo y sus dos pasadas Con **undo/redo logging** (registros **<T, X, t, v>**):

```
<START T1>
<T1, A, 5, 7>
<START T2>
<T2, B, 10, 11>
<COMMIT T1>
<T2, C, 12, 15>
<START T3>
<T3, D, 20, 25>
<COMMIT T3>
    <-- CRASH
```

- (a) Ejecute la **pasada de redo** (hacia adelante): ¿qué escrituras de transacciones confirmadas se re-aplican y a qué valor?
- (b) Ejecute la **pasada de undo** (hacia atrás): ¿qué escrituras de transacciones no confirmadas se deshacen y a qué valor?
- (c) Explique por qué se necesitan **dos** pasadas: dé un ejemplo concreto, dentro de este log, de (i) una transacción confirmada cuya escritura podría no estar en disco y (ii) una no confirmada cuya escritura sí podría estarlo.

Sugerencia. La flexibilidad de undo/redo (no forzar el orden datos/commit) es justamente lo que obliga a cubrir ambos casos: por eso el log guarda *ambos* valores.

Ejercicio 4. Orden de escrituras: ¿legal o ilegal? Para cada secuencia de eventos físicos, diga si **viola** la regla de logging indicada. Use: $\log(r)$ = el registro r se escribe en el buffer de log; $\text{flush_log}(r) = r$ ya está en disco; $\text{write}(X)$ = la página con la nueva versión de X se escribe a disco.

- (a) **Undo (regla U1, WAL).** `write(A) ; log(<T,A,old>) ; flush_log(<T,A,old>)`
- (b) **Undo (regla U2a).** `log(<T,A,old>) ; flush_log ; log(<COMMIT T>) ; flush_log(<COMMIT T>) ; write(A)`
- (c) **Redo (regla R1).** `log(<T,A,new>) ; log(<COMMIT T>) ; flush_log ; write(A)`
- (d) **Redo (regla R1).** `log(<T,A,new>) ; write(A) ; log(<COMMIT T>) ; flush_log`

Para cada caso ilegal, describa un crash en el peor momento que deje la base en un estado del que *no* se puede recuperar.

Sugerencia. En undo, los datos modificados deben estar en disco *antes* del commit; en redo, el log con `<COMMIT T>` debe estar en disco *antes* que cualquier dato de esa transacción.

Ejercicio 5. Checkpoints no-quiescentes Considere el siguiente log con **undo logging** y un checkpoint no-quiescente:

```
<START T1>
<T1, a, 5>
<START T2>
<T2, b, 10>
<START CKPT (T1, T2)>
<T2, c, 15>
<START T3>
<T1, d, 20>
<COMMIT T1>
<T3, e, 25>
<COMMIT T2>
<END CKPT>
<T3, f, 35>
      <-- CRASH
```

- (a) ¿Qué transacciones hay que deshacer, y hasta qué punto del log es necesario retroceder?
- (b) Ahora suponga que el crash ocurrió **antes** de escribir `<END CKPT>` (justo después de `<COMMIT T2>`). ¿Cambia su respuesta? ¿Hasta dónde hay que retroceder ahora y por qué?

Sugerencia. Si aparece `<END CKPT>` primero, todas las transacciones activas al `<START CKPT>` ya terminaron y sus datos están en disco; basta con deshacer las incompletas posteriores. Si aparece `<START CKPT>` sin su `<END CKPT>`, hay que retroceder al checkpoint *anterior* completo (o al `START` de la transacción incompleta más antigua).

3. Búsqueda de Texto

Recordatorio. Un **índice invertido** mapea cada *término* a su **posting list** (la lista ordenada de documentos donde aparece). Antes de indexar, el texto pasa por el pipeline *tokenización* $\rightarrow$ *normalización* $\rightarrow$ *stop words* $\rightarrow$

stemming. Para rankear usamos

$$\text{TF}(t, d) = \# \text{veces que } t \text{ aparece en } d, \quad \text{IDF}(t) = \log_{10} \frac{N}{\text{df}(t)}, \quad \text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \text{IDF}(t),$$

con N = número de documentos del corpus y $\text{df}(t)$ = número de documentos que contienen t .

Ejercicio 1. Construir un índice invertido Considere el corpus (cada línea es un documento):

d1: "Las bases de datos relacionales son fundamentales"

d2: "Los datos no estructurados crecen rapido"

d3: "Bases de datos NoSQL y datos relacionales"

- (a) Aplique el pipeline asumiendo: stop words {las, los, de, no, son, y}; normalización a minúsculas y sin acentos; stemming simplificado **bases**->**base**, **datos**->**dato**, **relacionales**->**relacional**, **estructurados**->**estructurado**, **crecen**->**crec**. Escriba la lista de términos resultante para cada documento.
- (b) Construya el índice invertido: para cada término, su posting list ordenada por doc-id.
- (c) ¿Por qué conviene mantener las posting lists *ordenadas por doc-id*? Conéctelo con el algoritmo de intersección.
- (d) ¿Por qué un B+ tree común sobre la columna **contenido** **no** sirve para responder “¿qué documentos hablan de *datos*?”?

Sugerencia. Tokenización “abre” (texto → palabras) y stemming “cierra” (variantes → raíz); ambas existen para que **bases** y **base** terminen en la *misma* posting list.

Ejercicio 2. Búsqueda booleana e intersección Dadas las posting lists (ya ordenadas por doc-id):

```
base    -> [1, 3]
dato    -> [1, 2, 3]
nosql   -> [3]
crec    -> [2]
```

- (a) Resuelva **base AND dato**, **dato OR crec** y **dato AND NOT base** operando sobre las listas. ¿Qué operación de conjuntos corresponde a cada uno?
- (b) Para una consulta **A AND B AND C** donde **posting(A)=120**, **posting(B)=8.000**, **posting(C)=12.000** entradas, ¿en qué orden conviene intersectar y por qué? Relacione con el principio de “empujar las selecciones más restrictivas hacia abajo”.

Sugerencia. El merge lineal avanza dos punteros comparando cabezas; los skip pointers permiten “saltar” bloques de la lista larga cuando el valor buscado está lejos.

Ejercicio 3. Búsqueda de frases con posiciones Las posting lists ahora guardan **posiciones**:

término -> [docID@pos, ...].

```
bases    -> [d3@1, d5@8, d9@2]
datos    -> [d3@2, d5@40, d9@3]
```

- (a) ¿Qué documentos retorna **bases AND datos** (sólo intersección de doc-ids)?
- (b) ¿En cuáles aparece la **frase exacta "bases datos"** (palabras consecutivas)? Muestre el chequeo de posiciones.

Sugerencia. Tras intersectar doc-ids, una frase de dos palabras existe en d si hay posiciones $p, p+1$ tales que la primera palabra está en p y la segunda en $p+1$.

Ejercicio 4. TF-IDF paso a paso Corpus de **4 documentos** (ya tokenizados y con stemming):

d1: [perro, perro, gato]
d2: [perro, casa, casa]
d3: [gato, casa, ratón, perro]
d4: [perro, perro, perro, casa]

Consulta $q = \{\text{gato, casa}\}$.

- (a) Calcule $\text{TF}(t, d)$ de **gato** y **casa** en cada documento.
- (b) Calcule $\text{df}(t)$ e $\text{IDF}(t) = \log_{10}(N/\text{df})$ para **gato** y **casa** con $N = 4$.
- (c) Calcule el score $s(q, d) = \sum_{t \in q} \text{TF}(t, d) \cdot \text{IDF}(t)$ para los cuatro documentos y entregue el ranking.
- (d) **perro** aparece en todos los documentos. ¿Cuánto vale su IDF? Explique por qué eso *no* es un error sino el principio central de TF-IDF.

Sugerencia. TF-IDF no premia “más palabras” sino “más palabras *raras*”: un término que aparece en todos los documentos no discrimina, y su peso es exactamente 0.

Ejercicio 5. Modelo vectorial y cosine similarity Usando dos términos como dimensiones, [gato, casa], y los valores TF-IDF del ejercicio anterior:

- (a) Escriba el vector TF-IDF de d_2 y d_3 en el espacio [gato, casa].
- (b) La consulta también es un vector: como $\text{TF}(t, q) = 1$ para sus términos, $q_t = \text{IDF}(t)$. Escriba el vector de q .
- (c) Calcule $\cos(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$ para d_2 y d_3 . ¿Cuál es más similar a la consulta?
- (d) Explique en una frase por qué la división por $\|d\|$ evita que un documento muy largo gane “por defecto”.

Sugerencia. El producto punto premia coincidencias; la normalización por las normas controla el largo, de modo que el score mide *dirección* y no *magnitud*.

Ejercicio 6. BM25 y PostgreSQL

- (a) TF-IDF tiene dos problemas que BM25 corrige: el TF *crece sin freno* y los *documentos largos ganan por defecto*. Para cada uno, indique qué parte de la fórmula de BM25

$$\text{BM25}(t, d) = \text{IDF}(t) \cdot \frac{\text{TF}(t, d) (k_1 + 1)}{\text{TF}(t, d) + k_1 (1 - b + b \frac{|d|}{\text{avgdl}})}$$

lo resuelve, y qué hace el parámetro asociado (k_1 o b).

- (b) Con $k_1 = 1,2$, el factor TF de BM25 satura cerca de $k_1 + 1 = 2,2$. ¿Por cuánto crece el aporte de un término al pasar de TF = 1 a TF = 100 en TF-IDF puro, versus en BM25? ¿Qué intuición de relevancia captura esa saturación?
- (c) Escriba el SQL para (i) crear un índice invertido sobre `articulos(titulo, cuerpo)` usando el índice GIN y el comeando `to_tsvector('spanish', ...)`, y (ii) una consulta que use `to_tsquery` y el operador `@@` para encontrar artículos sobre “bases” y “datos”.
- (d) ¿Por qué *ambos* —al indexar y al consultar— deben pasar por el mismo pipeline (`to_tsvector` / `to_tsquery`)? ¿Qué pasa si normaliza distinto en cada lado?
- (e) Explique brevemente la diferencia entre un vector **sparse** (BM25/TF-IDF, una dimensión por término) y uno **dense** (embedding, ~768 dimensiones latentes). ¿Por qué el índice invertido sirve para el primero pero no para el segundo (que usa HNSW/IVF)?

Sugerencia. BM25 es, geoméricamente, “TF-IDF con dos manijas”: k_1 satura el TF y b penaliza el largo. `ts_rank` de PostgreSQL *no* es TF-IDF completo porque no usa el df global del corpus.

4. Data Warehousing y SQL Analítico

Recordatorio. Un **Data Warehouse** separa la carga transaccional (OLTP, muchas escrituras pequeñas) de la analítica (OLAP, pocas consultas enormes con agregaciones). Se modela con un **esquema estrella**: una *tabla de hechos* (sólo llaves foráneas + métricas numéricas) rodeada de *tablas de dimensión* desnormalizadas. Los datos llegan por un proceso **ETL** (Extract, Transform, Load).

Ejercicio 1. Diseñar un esquema estrella Una aerolínea tiene este esquema OLTP normalizado (BCNF):

```
Pasajeros(pid, nombre, email, ciudad, id_pais)
Países(id_pais, nombre, continente)
Vuelos(vid, origen, destino, id_aerolinea, fecha_hora)
Aerolineas(id_aerolinea, nombre, alianza)
Aeropuertos(codigo, ciudad, id_pais)
Tickets(tid, pid, vid, clase, precio, fecha_compra)
```

Se quiere analizar **ingresos por país del pasajero, aerolínea, clase y mes de compra**.

- (a) Proponga una **tabla de hechos** `hechos_tickets`. ¿Cuál es el “grano” (qué representa una fila)? ¿Qué columnas son llaves foráneas y cuáles son métricas?
- (b) Diseñe las **tablas de dimensión** (`dim_pasajero`, `dim_aerolinea`, `dim_fecha`, ...). Incluya en cada una los atributos descriptivos que el OLTP tenía repartidos en varias tablas.
- (c) Escriba la consulta analítica pedida (ingresos por país, aerolínea, clase y mes) sobre su esquema estrella. ¿Cuántos joins necesita, comparado con cuántos exigía el esquema OLTP?

Sugerencia. En la dimensión, “aplane” las cadenas de joins del OLTP: `Pasajeros -> Países` se vuelve un solo `dim_pasajero` con `pais` y `continente` ya incluidos.

Ejercicio 2. Desnormalización y BCNF Después de una revisión, ven que la tabla de dimensión dada por `dim_pasajero(id, nombre, ciudad, region, pais, continente)` tiene las dependencias funcionales

$ciudad \rightarrow region, \quad region \rightarrow pais, \quad pais \rightarrow continente,$

que son transitivas y por lo tanto **violan BCNF**. Como las dimensiones son desnormalizadas, el contenido de `ciudad`, `region`, `pais` y `continente` se **repite** en cada fila de `pasajero`. Por ejemplo:

id	nombre	ciudad	region	pais	continente
1	Ana	Santiago	Metropolitana	Chile	América
2	Beto	Santiago	Metropolitana	Chile	América
3	Carla	Valparaíso	Valparaíso	Chile	América
4	Diego	Punta Arenas	Magallanes	Chile	América

- ¿Qué pasa si** la región **Metropolitana** se renombra oficialmente a **RM** y la tabla tiene 50.000 pasajeros de Santiago? ¿Cuántas filas hay que modificar? ¿Y si por un error de ese `UPDATE` sólo se cambian la mitad: qué situación inconsistente queda almacenada?
- ¿Qué pasa si** queremos registrar que la ciudad **Iquique** pertenece a la región **Tarapacá** (país **Chile**), pero *todavía no tenemos ningún pasajero* de Iquique? ¿Podemos guardar ese hecho en `dim_pasajero` tal como está definida? ¿Por qué?
- ¿Qué pasa si** se borra a Diego (fila `id=4`), que es el único pasajero de Punta Arenas? ¿Qué información, no relacionada con Diego, se pierde sin querer?
- Los tres problemas anteriores son justamente lo que la normalización a BCNF evita, y serían graves en un sistema OLTP donde *usuarios editan filas una a una*. Explique por qué en un Data Warehouse casi no importan: ¿cómo y cuándo se cargan y modifican las tablas de dimensión? Dé además dos ventajas concretas de *no* normalizar la dimensión para el workload analítico.
- Una alternativa es el **snowflake schema**: normalizar la dimensión en `dim_ciudad -> dim_region -> dim_pais`, eliminando la repetición. ¿Qué se gana y qué se pierde? ¿Por qué la mayoría prefiere la estrella?

Sugerencia. Note que cada “¿qué pasa si...?” nace de que un mismo hecho (“Santiago está en la Metropolitana”) está repetido o atado a la existencia de un pasajero. En un DW las dimensiones se (re)cargan en *bloque* por el ETL, no se editan fila a fila desde una aplicación, así que el riesgo que justificaba BCNF no llega a materializarse. El diseño correcto depende del *workload*.

Ejercicio 3. ROLLUP y CUBE Con el esquema estrella, la consulta base agrupa ventas por `region` y `familia`:

```
SELECT c.region, p.familia, SUM(h.monto) AS total
FROM hechos_ventas h
JOIN dim_cliente c ON h.id_cliente = c.id_cliente
JOIN dim_producto p ON h.id_producto = p.id_producto
GROUP BY ...;
```

Suponga el detalle: (Norte, Electrónica, 150), (Norte, Ropa, 80), (Sur, Electrónica, 120), (Sur, Ropa, 60) (en miles).

- Reemplace el `GROUP BY` por `GROUP BY ROLLUP(c.region, p.familia)` y escriba **todas** las filas del resultado, incluyendo los `NULL` de subtotal. ¿Cuántos niveles de agrupación genera?

- (b) Haga lo mismo con `GROUP BY CUBE(c.region, p.familia)`. ¿Qué filas adicionales aparecen respecto a `ROLLUP`?
- (c) Con $n = 3$ dimensiones, ¿cuántas agrupaciones genera `ROLLUP`? ¿Y `CUBE`? Dé la fórmula general.
- (d) En la salida, un `NULL` en la columna `region` puede significar “subtotal” o un valor nulo real del dato (un cliente sin región cargada). ¿Qué función SQL los distingue y qué retorna en cada caso?

Sugerencia. `ROLLUP(A,B)` asume una jerarquía (de lo grueso a lo fino) y produce $n + 1$ agrupaciones; `CUBE` produce 2^n (todas las combinaciones). `GROUPING(col)` vale 1 si la columna fue agregada en esa fila.

Ejercicio 5. ETL / ELT y la etapa Transform El DW se carga desde dos sistemas OLTP. Aparecen estos datos crudos:

Sistema A: `cliente_id=1042, rut="12.345.678-9", email="juan@ejemplo.cl", region="Región Metropolitana"`

Sistema B: `customer_id=8831, rut="123456789", email="JUAN@EJEMPLO.CL", region="R.M.", fecha_nac="32/13/1990", monto=-500`

- (a) Identifique al menos **cuatro** problemas de calidad de datos en estos registros.
- (b) Para cada problema, indique qué regla de la etapa **Transform** lo corrige (deduplicación, estandarización, validación, enriquecimiento o agregación).
- (c) ¿Los dos registros corresponden al mismo cliente? ¿Qué evidencia usaría para decidirlo y qué haría el Transform al respecto?
- (d) Explique la diferencia entre **ETL** (Extract-Transform-Load) y **ELT** (Extract-Load-Transform). ¿Qué cambio tecnológico (almacenamiento barato, cómputo elástico) hizo popular al ELT?

Sugerencia. “Garbage in, garbage out”: la calidad del DW depende casi por completo del Transform. En ELT se cargan los datos crudos primero y se transforman *dentro* del motor, on-demand y de varias formas para distintos usos.

Ejercicio 6. Cubo, operaciones OLAP y vistas materializadas

- (a) Para cada operación OLAP, escriba a qué construcción SQL corresponde sobre el esquema estrella: **roll-up**, **drill-down**, **slice**, **dice**.
- (b) Escriba la sentencia para crear una **vista materializada** `ventas_region_familia` que pre-calcule `SUM(monto)` por región y familia. ¿En qué se diferencia de una vista normal, y cuándo conviene usarla?
- (c) ¿Por qué hay que *refrescar* una vista materializada, y qué comando lo hace? ¿Qué riesgo introduce respecto a la frescura de los datos?
- (d) Conceptual: el *star schema* “no es obsoleto” aunque hayan aparecido data lakes y lakehouses. Explique en dos o tres frases por qué el modelo dimensional sigue siendo el lenguaje de referencia, sea el motor Postgres, Snowflake o DuckDB.

Sugerencia. Roll-up/drill-down cambian el nivel del `GROUP BY`; slice fija una dimensión (`WHERE año=2025`) y dice filtra varias a la vez. Las operaciones OLAP no son magia: son el vocabulario SQL de siempre aplicado a un modelo dimensional.